

ZedraI · M3 — Shopfloor Planning & Management

Developer Handover · Build-ready specification set · v1

Z Company · ZedraI

2026-06-03

Table of Contents

ZedraI · M3 Shopfloor Planning & Management — Developer Handover

00 · Master Index, Architecture, Stack & Conventions

Audience: engineering team · **Status:** v1 for build (plan→schedule→execute; optimisation interfaces only) · **Date:** 2026-06-03

This is the build-ready specification set for **M3 · Shopfloor Planning & Management**, the platform’s planning-and-control module (the foundation layers — Data Lake, Canonical Model, Manifold, Unified Intelligence, Monitoring, Reporting, Financial Impact, Platform/Security — are specified in `../Dev_Handover/`; the first module, M2 Maintenance, is in `../M2_Maintenance_Intelligence/`). Read this index first: it gives module scope, the architecture, which **platform decisions (D1–D11)** bind M3, the **stack** M3 inherits, M3-specific conventions, and the **build roadmap**. The *why* behind every spec is the planning deep-plan one level up: `../M3_Shopfloor_Planning_Management_DeepPlan.md`.

1. Document set & reading order

#	Spec	Build priority
00	This index — architecture, stack, conventions, roadmap	Read first
01	Data Model & Schema — new canonical entities, DDL dictionary (M3_schema.sql),	Phase M3-0

02	posture switch PPC, Planning Hierarchy & Postures — demand→MPS→MR P→CRP; the 3 postures	Phase M3-2→3
03	Detailed Scheduling & Sequencing — finite capacity, setup matrix, campaign rules, what-if	Phase M3-2
04	Shift Handover & Execution — handover lifecycle, carryover, confirmations (M1 graduation)	Phase M3-1
05	Production-Order Lifecycle & Events — state machine, domain events, M1 bootstrap	Phase M3-1
06	KPIs, Analytics & Financial Impact — formulas + SQL, registry, M3→M4 reconciliation	Phase M3-2
07	APIs, Events & Integration — REST/OpenAPI, domain events, SAP PP connector mapping	Phase M3-1→3
08	Optimisation / AI — Future-Scope Interfaces — optimiser hooks, readiness gate (no v1 build)	Phase M3-4 (gated)
09	Security, RBAC,	Cross-cutting

	NFRs & Acceptance — roles, multi-tenancy, test plan, acceptance	
10	Glossary — planning & control terms	Reference
—	M3_schema.sql	Runnable PostgreSQL DDL (paired with 01)

Companion material: the planning deep-plan (../M3_Shopfloor_Planning_Management_DeepPlan.md) + 10 flowcharts (../M3_diagram_*.mermaid). Platform specs referenced throughout live in ../../Dev_Handover/.

2. Module scope & non-goals

In scope (v1): planning master data (material, BOM, routing, work-centre capacity, setup matrix); the **posture-configurable** PPC engine (ingest *or* generate demand—MPS→MRP—orders); detailed **finite-capacity scheduling & sequencing** with sequence-dependent setup; dispatch & the **production-order lifecycle; execution, confirmation & shift handover** (graduating M1's first-class handover); the planning KPI family priced by Financial Impact; the live dispatch / plan-vs-actual board; ingestion from ERP/APS via Manifold **or** first-party planning **or** bootstrap from M1's SAP-PP CSV + actuals.

Future scope (interfaces only, build gated — D9): mathematical **schedule optimisation** (minimise setup+tardiness+WIP), **ML demand forecasting**, automated reactive re-scheduling, prescriptive recommendations. v1 ships the data hooks and interface stubs (spec 08) so this is a later switch-on, not a re-model.

Non-goals (neighbouring modules/systems): the financial ledger (M3 *feeds* Financial Impact, doesn't own cost accounting); warehouse execution/WMS (putaway, bin management — M3 reads stock for MRP netting only); quality disposition (M6 owns it; M3 reads quality holds for handover); maintenance scheduling (M2 owns it; M3 consumes PM windows as capacity constraints).

3. System context

flowchart LR

```

subgraph SRC["Planning in (ingest OR generate)"]
  ERP["ERP/APS (SAP PP) via Manifold"]
  M1P["M1 SAP-PP CSV (plan seed) + prod_* actuals + handover"]
  CAP["M3 first-party planning (PWA)"]
  CON["M2 PM windows + M4 state → constraints"]
end

```

```

subgraph M3SVC["M3 services"]
  MD["Master-Data svc (material/BOM/routing/capacity/setup)"]
  PPC["PPC svc (posture-gated MPS/MRP/CRP)"]
  SCH["Scheduler svc (finite-capacity + sequence)"]
  OE["Order/Dispatch svc (lifecycle)"]
  EX["Execution/Handover svc"]
  KPI["KPI/Adherence svc"]
  OPT["Optimiser (future, stub)"]
end
CANON["canon.* (Event Spine + ops.* schema)"]
SERVE["Serving zone / KPI mart / Redis live cache"]
subgraph CONS["Consumers"]
  UIL["Unified Intelligence"]
  MON["Operational Monitoring"]
  FIN["Financial Impact"]
  REP["Reporting"]
  M4["M4 OEE (shared ProductionCount)"]
end
ERP --> PPC
M1P --> PPC
M1P --> EX
CAP --> MD
CON --> SCH
MD --> PPC --> SCH --> OE --> EX
EX -- actuals --> PPC
OE & EX --> KPI
OPT -.future.-> SCH
MD & PPC & SCH & OE & EX & KPI --> CANON --> SERVE
SERVE --> UIL & MON & FIN & REP & M4

```

Golden rule (inherited): M3 reads canonical data through the Serving zone and writes its facts back through the canonical write-back API + ops.* schema. It never reads another module's database and never writes canon core tables it does not own. A produced quantity is **one** canon.event ProductionCount row — M3 contextualises it (attainment), M4 aggregates it (OEE).

4. Binding decisions that constrain M3 (subset of D1-D11)

#	Decision	M3 build constraint
D1	Hybrid-ready, per-client	Scheduler + dispatch + handover capture must run on-prem edge (plants are often disconnected); containerized, no cloud-only calls.
D2	Near-real-time (minutes), tunable	Plan-vs-actual & dispatch board refresh in minutes (per-tenant); confirmation capture never blocks.
D4	Excel/CSV + generic DB connector first	The M1 SAP-PP plan CSV is the first ingest path; SAP PP API connector is a fast-follow (spec 07).
D7	Cost-rates client-owned,	Late-order / idle / changeover costing resolves the

	effective-dated, “rate as of”	canon.cost_rate effective at the event date , never “today’s” rate.
D8	Logical tenant isolation default	tenant_id on every ops.* row; all queries tenant-scoped at the data-access layer.
D9	Rule-based now, ML later	Sequencing is deterministic/rule-based in v1 (setup matrix + campaign rules); optimisation ML is reserved (spec 08), not built.
D11	Sector-neutral, exercise extension namespace	Order/routing/capacity/setup model is generic; Hero Steels is a <i>seed template</i> , not the schema. Client specifics live in ext JSONB.

Full D1–D11 text: ../../Dev_Handover/00_INDEX_and_Architecture.md §3.

5. Technology stack (inherited + M3-relevant)

M3 adds **no new platform technology**; it uses the stack ratified in the platform handover.

Concern	Tech	M3 use
Canonical OLTP	PostgreSQL 15+ (ops schema)	all M3 master + transactional tables (M3_schema.sql)
KPI mart	ClickHouse / Postgres columnar	kpi_plan_daily rollups for UIL/Reporting
Event bus	Kafka / Redpanda	publish/consume planning domain events (spec 07)
Scheduler / batch	Apache Airflow	MRP run, capacity rollup, KPI rollup, plan-vs-actual jobs
Live cache	Redis	dispatch board, plan-vs-actual, late-order counts (Monitoring)
API	REST + OpenAPI , gateway, OIDC	spec 07 endpoints
Auth/RBAC	Keycloak (OIDC)	M3 roles extend M1 (spec 09)
Backend	Python 3.11 + FastAPI	M3 services; scheduler may use OR-Tools later (spec 08)
Frontend	React + TS (PWA)	planner board, dispatch, handover; reuse M1 capture patterns
Optimiser (later)	OR-Tools / metaheuristics	reserved for §8, not built in

6. M3 build roadmap (epics)

flowchart TD

M0["M3-0 · Foundations
ops schema · material/BOM/routing/capacity/setup · planning_config posture · RBAC roles"]

M1["M3-1 · Execute & Hand over (closest to M1)
production-order lifecycle · confirmations · shift handover graduation · M1 plan-CSV + actuals bootstrap"]

M2x["M3-2 · Schedule & Measure
finite-capacity scheduler + setup/campaign sequencing + what-if · KPI engine (attainment/OTIF/throughput) · Financial Impact · monitoring board"]

M3["M3-3 · Plan & Integrate
posture A/B ingest via Manifold SAP PP connector · MRP/MPS engine for posture C · reporting pack"]

M4["M3-4 · Optimise (GATED on data + D9)
sequence optimiser · ML demand forecast · reactive re-schedule · OR-Tools/MLflow"]

M0 --> M1 --> M2x --> M3 --> M4

Maps onto the platform roadmap: M3-0→2 ride Phase 2–3 (serve/synthesize); M3-3 rides Phase 4 (SAP/connectors); M3-4 rides Phase 5 (ML/optimisation, data-gated). Note the **on-ramp order** (execute-first) deliberately front-loads the cheapest value (M1 already supplies the plan CSV, actuals, and a handover).

7. Non-functional baseline (delta over platform \$7)

M3 inherits the platform NFR baseline (multi-tenant, hybrid, minutes-latency, 99.5% serving, OIDC, audit, OTel). Module-specific targets (detailed in spec 09): the scheduler produces a committed sequence for ≥10k open operations/tenant within interactive time; first-party confirmation/handover capture is **offline-tolerant** (queue locally, sync on reconnect) like M1; KPI rollups reconcile with M4 to the produced-unit; handover sign-off is an immutable, audit-logged record.

8. Open items (carry into build)

- **Default posture** per segment (lead with Posture B “own-the-schedule”?) — spec 02.
- **Scheduling granularity**: work-centre/shift bucket vs machine/minute — spec 03 (planning_config.schedule_grain).
- **Setup-matrix source**: planner-entered vs learned from M1 actual changeover times — specs 03/08.
- **MRP scope (Posture C)**: single- vs multi-level explosion; net-change vs regenerative; purchase reqs or make-orders only — spec 02.
- **Material/stock ownership**: M3 holds balances for netting vs always reads ERP/WMS — specs 02/01.

- **Handover sign-off:** single vs dual (outgoing+incoming) acknowledgement gate — spec 04.
- **Optimisation data gate:** history threshold that flips a work centre to “optimisation-ready” — spec 08.
- **D11:** whether M3 (a discrete/assembly client) is used to validate the canonical model beyond steel.

Next: specs 01–10. The compiled Word/PDF package combines all of these into one M3 Developer Handover document.

M3 • 01 — Data Model & Schema Spec

Pairs with: M3_schema.sql (runnable PostgreSQL 15+). **Deep-plan ref:** §11 (data contract), §5 (postures), §12 (boundaries).

1. Where M3 sits in the canonical model

M3 owns the ops schema and **FKs into canon.***; it never alters canon core. It **brings to life** four canonical entities that were catalogued (doc 01 §10–11) but not yet built (no prior module needed them): Material, BOM, ProductionOrder, ProductionOperation. It **introduces** (additive, doc 01 §14): Routing/RoutingOperation, WorkCenterCapacity, SetupMatrix, DemandForecast, MPSLine, PlannedOrder, ScheduleEntry, ProductionConfirmation, ShiftHandover/HandoverCarryover, MaterialStock, MrpRun, kpi_plan_daily.

2. Conventions (inherited from 02_CanonicalDataModel_Spec §5)

lower_snake_case · surrogate BIGINT GENERATED BY DEFAULT AS IDENTITY · UUID tenant_id on every row · TIMESTAMPTZ UTC · units embedded in column names · ext JSONB extension namespace · source_ref provenance on ingested rows · FK to canon.* masters (no free-text coded values).

3. The soft-reference rule (canon independence)

canon.event.order_ref is a plain BIGINT (doc 02 §5 DDL — no hard FK). M3 places ops.production_order.order_id into it. **No cross-schema hard FK from canon→ops**, so the canonical core stays independent of M3 — the same rule M2 uses for downtime classification. Likewise ops.production_confirmation.canon_event_ref softly points at the canon.event ProductionCount row M3 books.

4. Entity dictionary (highlights — full DDL in M3_schema.sql)

Table	Grain	Key columns	Notes
planning_config	1 / tenant	posture, schedule_grain, mrp_enabled, optimiser_enabled	the posture switch (§5); drives which engines run
material	material	sku, material_type, base_uom, grade_ref	canonical master; grade_ref maps to M1 grade
bom_line	parent×component	qty_per, scrap_pct	MRP explosion input
routing / routing_operation	material route	seq_no, work_center_ref, run_minutes_per_unit, setup_minutes_std	the path orders follow; FK to canon.equipment_node
work_center_capa	WC × shift × period	basis,	finite-capacity

city		available_minutes/ available_qty, efficiency_pct, effective-dated	supply
setup_matrix	WC × from→to	setup_minutes, setup_cost, is_forbidden, rule_note	sequence- dependent setup (steel wide→narrow); optimiser substrate
demand_forecast	demand line	demand_type, qty, demand_date	ingested (A/B) or generated (C)
mps_line	material × period	planned_qty	master schedule
mrp_run	run	run_type, counts	netting run log (Posture C)
material_stock	material × site	on_hand_qty, allocated_qty, reorder_point, lead_time_days	netting balances; source-owned in A/B
planned_order	planned order	planned_qty, need_date, is_firmed	MRP output
production_order	order	order_status, planned_qty, good_qty, due_ts, m1_coil_ref	the lifecycle spine; order_id carried by canon.event.order _ref
production_operation	order × op	seq_no, work_center_ref, operation_status	routing instance per order
schedule_entry	WC × sequence	sequence_no, planned_start_ts, setup_minutes, status, scenario_tag	finite-capacity schedule; scenario_tag = what-if
production_confirmation	confirmation	good_qty/ scrap_qty, setup_minutes, run_minutes, canon_event_ref	actuals; books a canon ProductionCount once
shift_handover	handover	status, planned_qty/prod uced_qty, oee_pct, sign-off ts	M1 graduation; plan-vs-actual feedback
handover_carryov	carryover line	kind,	the no-orphan

er		order_ref/asset_ref, is_mandatory, acknowledged	guarantee
kpi_plan_daily	WC × day	attainment/OTIF/throughput/util/setup + priced costs	mart for UIL/Reporting; reconciles to M4

5. Posture-driven population (same model, three doors)

The enum `order_source` (`ERP_IMPORT` | `MRP_GENERATED` | `MANUAL` | `M1_PLAN_CSV`) records *who authored* each demand_forecast/mps_line/planned_order/production_order row. In **Posture A/B** these are `ERP_IMPORT` (or `M1_PLAN_CSV`); in **Posture C** the MRP engine writes `MRP_GENERATED`. Downstream scheduling/execution/KPI code reads the same columns regardless — so posture is data, not a code fork (deep-plan \$5).

6. Acceptance criteria

- **MUST** create ops schema and all tables with `tenant_id` + ext JSONB on every row.
- **MUST** keep canon independent: no FK from `canon.*` to `ops.*`; `order_ref/canon_event_ref` are soft.
- **MUST** FK every coded reference to a master (`material_ref`, `work_center_ref`→`canon.equipment_node`, `shift_ref`→`canon.shift`, `*_ref`→`canon.personnel`).
- **MUST** record one `ProductionCount` per produced quantity (no double-book vs M4).
- **MUST** support all three postures without schema change (validated by populating each `order_source`).
- **SHOULD** pass M1 coil_plan/prod_* through conformance into `production_order/production_confirmation` with zero loss.
- **SHOULD** validate via `pglast.parse_sql` in CI (as M2 does).

7. Build phasing

M3-0: schema + masters + planning_config. **M3-1:** order lifecycle, confirmation, handover. **M3-2:** schedule_entry + setup_matrix + KPI mart. **M3-3:** demand/MPS/MRP + material_stock (Posture C). **M3-4:** optimiser tables stay as-is (no re-model).

M3 · 02 — PPC, Planning Hierarchy & Postures Spec (M3c)

Deep-plan ref: §2 (hierarchy), §5 (postures + MRP/MPS engine). **Standards:** APICS/ASCM MPC, MRP II, SAP PP object flow, ISA-95 Part 3.

1. The planning hierarchy M3c implements

Demand → MPS → RCCP → MRP → CRP → (detailed schedule, spec 03). Horizons shrink top→bottom (months→days). M3c owns the upper hierarchy **only to the depth the posture demands**; it always computes capacity load (RCCP/CRP) so it can flag an infeasible ingested plan even when it didn't author the plan.

2. The three postures (the defining design choice)

A single ops.planning_config.posture per tenant selects behaviour. **Same canonical entities, different author.**

Stage

Demand / forecast

MPS

MRP (explosion, netting)

RCCP / CRP (capacity)

Detailed scheduling (spec 03)

Dispatch + order lifecycle

Execution + handover (spec 04)

Adherence KPIs

Flags set per posture: A → mrp_enabled=false, schedule rows ingested with status='COMMITTED'. B → mrp_enabled=false, scheduler authors schedule. C → mrp_enabled=true, MRP engine authors demand/MPS/planned orders.

3. MRP/MPS engine (Posture C)

Standard deterministic run, triggered manually or by Airflow: 1. **MPS** — for independent-demand finished items, net demand_forecast against on-hand (material_stock) + scheduled receipts → mps_line. 2. **BOM explosion** — walk bom_line top-down; for each level compute **gross requirement** = parent planned qty × qty_per × (1 + scrap_pct). 3. **Netting** — net against material_stock.on_hand_qty − allocated_qty + scheduled receipts. 4. **Lead-time offset** — start = need date − lead_time_days → planned_order.need_date. 5. **Emit** — planned_order rows (source='MRP_GENERATED', mrp_run_ref set); make vs buy by material_type (FINISHED/WIP→make, RAW/CONSUMABLE→purchase requisition out via spec 07). 6. **CRP** — load resulting planned orders × routing_operation.run_minutes_per_unit against work_center_capacity; surface overload.

v1 = single- or multi-level explosion (config), **net-change** default with regenerative option (mrp_run.run_type). Multi-echelon/optimised MRP is future scope (spec 08).

4. RCCP/CRP capacity check (all postures)

required_minutes(WC, period) = Σ orders (run_minutes_per_unit $\times$ qty + setup_minutes); load_pct = required $\div$ (available_minutes $\times$ efficiency_pct/100).
load_pct > 100 $\rightarrow$ capacity alert (Monitoring) and an infeasibility flag on the ingested/generated plan. This diagnostic is valuable in **every** posture — it is how M3 tells a Posture-A client “your ERP’s plan can’t physically run on Tuesday.”

5. Interfaces (see spec 07 for full OpenAPI)

- POST /ppc/mrp-run (Posture C) $\rightarrow$ returns mrp_run_id, planned-order count.
- POST /ppc/mps — upsert MPS (ingest or generate).
- GET /ppc/capacity-load?work_center&from&to $\rightarrow$ load_pct series.
- POST /ppc/planned-orders/{id}/firm $\rightarrow$ firm a planned order ($\rightarrow$ eligible for release, spec 05).

6. Acceptance criteria

- **MUST** honour posture: in A/B the MRP engine is inert; in C it authors demand/MPS/planned orders.
- **MUST** compute capacity load in all postures and raise an infeasibility flag when load_pct > 100.
- **MUST** resolve cost in KPIs via cost_rate effective at event date (D7).
- **SHOULD** reconcile MRP gross-to-net arithmetic in a unit test against a worked BOM example.
- **SHOULD** import the **M1 SAP-PP plan CSV** as the first Posture-A/B source (spec 07 mapping).

M3 • 03 — Detailed Scheduling & Sequencing Spec (M3b)

Deep-plan ref: §6. **Standards:** APS / finite-capacity scheduling; ISA-95 “detailed production scheduling”; steel cold-rolling campaign rules.

1. Job

Turn firmed/released planned_order/production_order rows into an **executable finite-capacity sequence** (schedule_entry) per work centre, respecting capacity, material readiness, and **sequence-dependent setup**, minimising **setup + tardiness + WIP**.

2. Inputs

- Open orders + due dates + priority (production_order).
- Routing (routing_operation: WC, run-min/unit, std setup).
- Capacity supply (work_center_capacity by shift/period, efficiency).
- **Setup matrix** (setup_matrix: from_state→to_state minutes/cost, is_forbidden).
- Constraints: **M2 PM windows** (don't schedule on an asset in PM), **M4 live state** (resource down), material availability.

3. Algorithm (v1 — deterministic, rule-based)

1. **Filter** to schedulable operations (materials ready, predecessor done).
2. **Prioritise** by due date / priority (EDD default; critical-ratio optional).
3. **Sequence within a work centre** applying campaign rules: order by the setup-minimising attribute progression, honouring is_forbidden transitions; compute each entry's setup_minutes from setup_matrix given its predecessor's to_state.
4. **Place in time** forward (from now) or backward (from due) across work_center_capacity buckets; respect grain (planning_config.schedule_grain = WORK_CENTER_SHIFT | MACHINE_MINUTE).
5. **Detect overload / late** → flag; expose for planner override (drag-to-resequence).
6. **Commit:** set status='COMMITTED', scenario_tag=NULL. Prior committed rows for the horizon → SUPERSEDED.

4. Steel cold-rolling campaign rules (Hero Steels seed template)

Encoded as setup_matrix + sequence policy on the mill/rolling work centres: - **Width: wide → narrow** within a campaign (a wider coil marks the rolls and scratches subsequent narrower coils). - **Width increase forces a roll change** → high setup_minutes/setup_cost, or is_forbidden=true without an explicit roll-change entry. - **Gauge & hardness** progress smoothly (small step penalties). - Annealing furnaces scheduled by **charge** (capacity_basis='FURNACE_CHARGE', available_qty), not minutes. Other sectors supply their own setup_matrix + policy; the engine is unchanged (deep-plan §10).

5. What-if & re-scheduling

- **What-if:** write candidate schedule_entry rows under a scenario_tag; compare KPI deltas (setup total, tardy count, utilisation) before commit.
- **Reactive:** on breakdown / material short / hot order, re-run for the affected work centre(s) only; planner confirms. (Automated reactive re-schedule = future, spec 08.)

6. Interfaces

- POST /schedule/run?work_center&horizon[&scenario_tag] → schedule + KPI summary.
- POST /schedule/commit / POST /schedule/{id}/move (manual resequence).
- GET /schedule/dispatch-list?work_center&shift → v_dispatch_list.
- GET /schedule/whatif/compare?a&b → KPI delta.

7. Acceptance criteria

- **MUST** never exceed work_center_capacity in a committed schedule (finite-capacity guarantee).
- **MUST** compute setup_minutes from setup_matrix and never emit a is_forbidden transition uncoded.
- **MUST** treat M2 PM windows and M4 down-state as hard unavailability.
- **MUST** keep what-if (scenario_tag ≠ NULL) out of the committed dispatch list.
- **SHOULD** reproduce the steel wide→narrow campaign on the Hero Steels seed in an acceptance test.
- **SHOULD** store actual setup from confirmations (spec 04) to calibrate the matrix (feeds spec 08).

M3 • 04 — Shift Handover & Execution Spec (M3a)

Deep-plan ref: §7. **Standards:** HSE HSG48 human factors (structured, two-way, written+verbal); digital shift-logbook practice. **Scope note:** M3a serves the PPC loop (continuity + plan-vs-actual feedback), not a standalone safety product.

1. Two jobs

1. **Execution & confirmation** — run dispatched orders this shift; capture production_confirmation actuals (good/scrap/setup/run minutes), updating production_operation/production_order status.
2. **Shift handover** — at the boundary, compile a structured shift_handover that (a) carries every open order/WIP/stoppage forward (the **no-orphan guarantee**), (b) records **plan-vs-actual** for the shift, (c) feeds that variance back to planning (specs 02/03).

2. Handover lifecycle (ops.handover_status)

ACCUMULATING → DRAFTED → REVIEWED → SIGNED_OFF. Open orders/WIP/stoppages auto-accrue as handover_carryover while ACCUMULATING. DRAFTED auto-pulls actuals (no re-keying). REVIEWED enforces mandatory sections. SIGNED_OFF is immutable + audit-logged.

3. Mandatory, non-skippable sections (handover_carryover.is_mandatory)

Section	kind	Source
Open orders / WIP carryover	OPEN_ORDER / WIP	production_order still in progress
Running stoppages / degraded assets	STOPPAGE	M1 stoppages / M2
Safety: permits, isolations, temporary controls, incidents	SAFETY	first-party capture
Quality holds	QUALITY_HOLD	M1 defects / M6
Instructions / priorities for next shift	INSTRUCTION	supervisor

Header (shift in/out, supervisors, crew) + production-vs-plan (planned_qty/produced_qty/oeo_pct/variance_note) live on shift_handover.

4. Relationship to M1 (graduation)

M1 already carries open **coils** + running stoppages to the next shift log. M3a **generalises** this to any canonical order/WIP, adds the structured production-vs-plan + safety + instruction sections, enforces mandatory fields, and writes a canonical shift_handover. Where M1 exists, reuse its capture UX, **offline-tolerance**, and RBAC; where it doesn't, M3a

stands alone (works for non-M1 clients). The shift's produced_qty vs planned_qty is the planning **feedback signal** (spec 06 adherence; specs 02/03 re-plan).

5. Confirmation → canonical ProductionCount (the M3 ↔ M4 rule)

A confirmation books **one** canon.event ProductionCount (soft-linked via production_confirmation.canon_event_ref). M3 reads it for attainment; M4 reads it for OEE. Never written twice (deep-plan §12).

6. Interfaces

- POST /exec/confirm — body: order/op, good/scrap/rework, setup/run minutes, operator, shift → books ProductionCount + updates status.
- POST /handover (open) · POST /handover/{id}/draft|review|signoff.
- GET /handover/{id} → full structured handover (for the verbal+written meeting).
- GET /handover/open-carryover?area&shift → live no-orphan list.

7. Acceptance criteria

- **MUST** prevent SIGNED_OFF while any is_mandatory carryover is unacknowledged.
- **MUST** auto-carry every in-progress order (no orphan) into the draft.
- **MUST** make SIGNED_OFF immutable + audit-logged (who/when, both supervisors).
- **MUST** book exactly one ProductionCount per confirmed quantity.
- **MUST** be offline-tolerant (queue + sync) like M1.
- **SHOULD** auto-populate produced-vs-plan + OEE from M4 so the supervisor confirms, not types.

M3 · 05 — Production-Order Lifecycle & Events Spec

Deep-plan ref: \$4, \$11. **Standards:** ISA-95 Part 3 (dispatching, execution, tracking); SAP PP order→confirmation flow.

1. Lifecycle state machine (**ops.order_status**)

PLANNED → FIRMED → RELEASED → SCHEDULED → DISPATCHED →
IN_PROGRESS → COMPLETED → CLOSED

↘ (open at shift end) carried → resumes

PLANNED → CANCELLED (re-plan / demand drop)

- **PLANNED** — from MRP (Posture C) or ingest (A/B).
- **FIRMED** — planner fixes qty/dates (eligible for release).
- **RELEASED** — materials + capacity confirmed; release_ts set; order is now real work.
- **SCHEDULED** — placed in a committed schedule_entry (spec 03).
- **DISPATCHED** — on v_dispatch_list for a work centre/shift.
- **IN_PROGRESS** — first confirmation booked; run clock active.
- **COMPLETED** — all operations confirmed, good_qty booked, completed_ts set.
- **CLOSED** — variances settled & costed (Financial Impact); immutable.
- **CARRIED** is not a status but a handover relationship (spec 04) — an IN_PROGRESS order referenced by handover_carryover.

Transitions are validated server-side; illegal jumps rejected. Each transition emits a domain event (§3).

2. Operation-level tracking (**production_operation**)

Each order instantiates its routing into production_operation rows (PENDING→DISPATCHED→IN_PROGRESS→CONFIRMED). Tracking gives WIP location, genealogy (which op a unit is at), and — at Hero Steels — ties to the **M1 COIL_NO** spine via production_order.m1_coil_ref for end-to-end traceability.

3. Domain events (**Kafka topics — see spec 07**)

order.planned, order.firmed, order.released, order.scheduled, order.dispatched, order.started, production.confirmed, order.completed, order.closed, order.cancelled, handover.signed_off. Each carries tenant_id, ids, ts, lineage. Consumers: Monitoring (board), KPI svc, M4 (ProductionCount), Financial Impact, Reporting.

4. M1 bootstrap (**fastest first value**)

- planning.plan_order / coil_plan (M1's SAP-PP CSV) → production_order (source='M1_PLAN_CSV').
- prod_* capture (per-process output) → production_confirmation (m1_source_ref set) → ProductionCount.
- M1 shift_log handover → seed shift_handover (spec 04). This lights up the order lifecycle + adherence **before** any ERP connector exists.

5. Acceptance criteria

- **MUST** enforce the state machine; reject illegal transitions; audit every transition.
- **MUST** emit the domain event for each transition exactly once (idempotent producer).
- **MUST** support ingest (A/B) and MRP-generated (C) orders identically downstream.
- **MUST** preserve source_ref/lineage from M1/ERP on every order & confirmation.
- **SHOULD** expose order genealogy (operation-level WIP) for tracking & the M1 coil link.

M3 · 06 — KPIs, Analytics & Financial Impact Spec

Deep-plan ref: \$9, \$12. **Standards:** ISO 22400 (shared with M4); APICS adherence metrics.

1. KPI family (published to the shared registry, doc 03)

KPI	Formula	Notes
Schedule attainment	$\text{completed-on-schedule} \div \text{scheduled} \times 100$	planning discipline
Qty attainment	$\text{actual good} \div \text{planned} \times 100$	per order/WC/day
OTIF	$\text{on-time AND complete} \div \text{total orders} \times 100$	customer reliability
On-time delivery	$\text{on/before due} \div \text{total}$	
Throughput	$\text{good output} \div \text{time}$	
Capacity utilisation	$\text{load} \div \text{available} \times 100$	watch overload
Setup ratio	$\text{setup time} \div \text{available} \times 100$	+ changeover_count
Sequence compliance	$\text{jobs run in planned sequence} \div \text{total}$	steel campaign discipline
WIP / cycle time	$\text{release} \rightarrow \text{completion elapsed}$	lean flow
Schedule stability	$\text{plan changes} \div \text{horizon}$	nervousness
Forecast accuracy	1 -	fcst-act

2. Rollup

Airflow job aggregates production_confirmation + schedule_entry + work_center_capacity into kpi_plan_daily (WC × day). Served to UIL/Reporting; cached in Redis for the live board.

Example (attainment):

```
-- qty attainment per work centre per day
SELECT c.work_center_ref, date_trunc('day', c.confirmed_ts) AS d,
       SUM(c.good_qty) AS actual_good,
       SUM(po.planned_qty) AS planned,
       100.0 * SUM(c.good_qty) / NULLIF(SUM(po.planned_qty),0) AS
qty_attainment_pct
FROM ops.production_confirmation c
JOIN ops.production_order po ON po.order_id = c.order_ref
WHERE c.tenant_id = $1
GROUP BY 1,2;
```

3. The M3 ↔ M4 reconciliation (the line to get right)

A produced quantity is recorded **once** as a canon.event ProductionCount. **M3 contextualises** it (attainment, OTIF, sequence compliance — *did we run the plan?*); **M4 aggregates** it (OEE Performance/Quality — *how efficiently did the machine run?*). Two different metrics, one production truth. The same applies to StateEvent (M3 dispatch feasibility vs M4 Availability) and the Planned-Production-Time denominator (defined once by canon.shift). **Acceptance:** M3's throughput numbers must tie to M4's count inputs to the unit.

4. Financial impact (D7, effective-dated)

Price each gap via canon.cost_rate effective at the event date: - **Late-order / expedite cost** = late hours × expedite rate (or penalty per order). - **Idle-capacity cost** = unused available minutes × idle rate. - **Changeover cost** = setup_matrix.setup_cost × changeover count. - **WIP holding** = avg WIP value × holding rate. Written to kpi_plan_daily.late_order_cost / idle_capacity_cost and surfaced on exec reports — “attainment +8% = £X avoided expedite + idle.”

5. Acceptance criteria

- **MUST** reuse ISO 22400 definitions where they overlap M4 (no parallel truth).
- **MUST** resolve cost via the rate **effective at event date** (never today's).
- **MUST** reconcile throughput to M4 count inputs in a cross-module test.
- **SHOULD** publish every KPI to the shared registry with lineage to source rows.

M3 • 07 — APIs, Events & Integration Spec

Deep-plan ref: \$0, \$12. **Standards:** REST/OpenAPI; SAP PP object model (ingestion reference); ISA-95 Part 5 B2M transactions (framing).

1. REST surface (FastAPI + OpenAPI, OIDC, tenant-scoped)

Area	Endpoints (selected)
Master data	GET/POST /materials, /boms, /routings, /work-centers/{id}/capacity, /setup-matrix
PPC (spec 02)	POST /ppc/mrp-run, POST /ppc/mps, GET /ppc/capacity-load, POST /ppc/planned-orders/{id}/firm
Scheduling (spec 03)	POST /schedule/run, /schedule/commit, /schedule/{id}/move, GET /schedule/dispatch-list, GET /schedule/whatif/compare
Orders (spec 05)	POST /orders, POST /orders/{id}/release cancel, GET /orders/{id}, GET /orders?status&due
Execution (spec 04)	POST /exec/confirm, POST /handover, /handover/{id}/draft review signoff, GET /handover/open-carryover
KPIs (spec 06)	GET /kpis/plan-daily, GET /kpis/registry
Config	GET/PUT /planning-config (posture; admin-only, audit-logged)

All mutating endpoints are idempotent (client request_id); all reads tenant-scoped at the data layer (D8).

2. Domain events (Kafka/Redpanda)

Producers: order lifecycle (spec 05) + handover + schedule commit. Topics: m3.order.*, m3.production.confirmed, m3.schedule.committed, m3.handover.signed_off, m3.capacity.overload. Envelope: {tenant_id, event_id, ts, type, payload, lineage_ref}. Consumers: Monitoring, KPI svc, **M4** (ProductionCount), Financial Impact, Reporting, and **M1 write-back** (plan as auto-source).

3. Manifold connector mapping — SAP PP → canonical (Posture A/B ingest)

SAP PP object	Canonical / ops target	Notes
Material master (MARA/MARC)	ops.material	sku, type, base UoM
BOM (STKO/STPO)	ops.bom_line	qty_per, scrap
Routing (PLPO/MAPL)	ops.routing +	WC, std times

	routing_operation	
Work centre (CRHD) + capacity	canon.equipment_node + ops.work_center_capacity	
PIR / demand (PBED)	ops.demand_forecast	demand_type=PIR
MPS/MRP planned order (PLAF)	ops.planned_order	source=ERP_IMPORT
Production order (AUFK/AFKO/AFPO)	ops.production_order	order_no, qty, dates
Confirmation (AFRU)	ops.production_confirmation	actuals → ProductionCount

Field-mapping, dedup, master-data resolution, readiness all run in **Manifold** (doc 02). The **M1 SAP-PP plan CSV** is the *first* connector (D4) — same target tables, CSV instead of API.

4. Inbound constraints from sibling modules

- **M2** PM schedules → capacity unavailability windows (scheduler input, spec 03).
- **M4** live StateEvent → resource up/down for dispatch feasibility.
- **M6** quality holds → handover QUALITY_HOLD carryover.

5. Acceptance criteria

- **MUST** publish OpenAPI 3 for every endpoint; version independently of canon (doc 02 §7).
- **MUST** emit each domain event exactly once; consumers idempotent.
- **MUST** map the SAP PP objects above (and the M1 CSV) into the canonical/ops targets via Manifold with lineage preserved.
- **MUST** keep all reads/writes tenant-scoped (D8) and hybrid-edge runnable (D1).
- **SHOULD** ship the SAP PP connector map as a reusable **planning sector template** (doc 02 §12).

M3 · 08 — Optimisation / AI — Future-Scope Interfaces Spec

Deep-plan ref: §8. **Status:** interfaces & data hooks only — **no v1 build** (D9; operating-model post-6-month → prescriptive horizon, gated on data). This mirrors M2’s predictive spec: future-proof now, build later, no re-model.

1. What is deferred

Capability	v1 (built)	Future (this spec)
Sequencing	deterministic rules + manual resequence (spec 03)	mathematical optimisation: minimise setup+tardiness+WIP (MILP / metaheuristics, e.g. OR-Tools)
Demand	ingest / simple consumption	ML demand forecasting (seasonality/trend) → MPS
Re-scheduling	planner re-runs on disruption	automated reactive re-schedule within guardrails
Decision support	KPIs + alerts (descriptive/diagnostic)	prescriptive (“run this sequence; expedite order X”), 2–3 yr

2. Why v1 is already the substrate (no re-model)

The optimiser’s inputs **already exist** from day one: `setup_matrix` (transition costs), `work_center_capacity` (constraints), `routing_operation` (times), `production_order` (due/priority), `canon.cost_rate` (objective coefficients), and — critically — `production_confirmation.setup_minutes/run_minutes` (the **actuals** that calibrate standards and become training/validation data). Switching optimisation on is wiring an engine to existing tables, not migrating data.

3. Interface stubs (defined, not implemented)

- `POST /optimise/schedule` → same contract as `/schedule/run` but returns an objective-minimising sequence + objective value; behind `planning_config.optimiser_enabled`.
- `GET /optimise/readiness?work_center` → the **optimisation-readiness gate**.
- `POST /forecast/run` → ML demand forecast → `demand_forecast` (source tagged `MODEL`).
- Feature store (Feast) + model registry (MLflow) reserved (platform stack).

4. Optimisation-readiness gate (History sub-gate)

Extends the platform readiness model (doc 01 §16). Per work centre, report whether there is **enough confirmed run/setup history** to trust an auto-schedule (e.g. ≥ N weeks of confirmations covering the product/transition mix). The UI shows “optimisation available

for these work centres” — honest readiness, **no v1 optimisation promise**. Threshold = open decision (spec 00 §8).

5. Acceptance criteria (for the v1 stubs)

- **MUST** gate all optimiser endpoints behind `optimiser_enabled=false` by default (return 501/not-enabled).
- **MUST NOT** require any schema change to enable optimisation later (validated by review against `M3_schema.sql`).
- **MUST** persist actual setup/run minutes on every confirmation (the calibration data) from v1.
- **SHOULD** expose the readiness gate read-only in v1 so the roadmap is visible without promising delivery.

M3 · 09 — Security, RBAC, NFRs & Acceptance Spec

Deep-plan ref: §13. **Inherits:** platform 08_Platform_Security_Spec + M1 RBAC.

1. Roles (extend M1: Operator / Supervisor / Plant-Head / Admin)

Role	Can
Production Planner	MPS/MRP, posture config requests, firm planned orders, what-if
Scheduler	run/commit detailed schedule, resequence, dispatch
Shift Supervisor	execute dispatch, confirm production, own & sign off shift handover
Operator	confirm own work centre/line (reuse M1 capture), acknowledge carryover
Materials Controller	stock/MRP netting (Posture C)
Plant Manager	read-only KPIs, attainment, cost
Admin	masters, integration, posture & workflow config (audit-logged)

Row-level scoping by site/area (reuse M1 line_access). Posture is admin-only and audit-logged.

2. Security & multi-tenancy

OIDC (Keycloak); tenant_id on every ops.* row; queries tenant-scoped at the data layer (D8); logical isolation default, physical on request. Handover sign-off and order state transitions are immutable, fully **audit-logged** (who/when/what) — handover is a legal-grade record (HSE).

3. NFRs (delta over platform §7)

Concern	Target
Hybrid/edge (D1)	scheduler + dispatch + capture run on-prem disconnected; sync on reconnect
Latency (D2)	dispatch board + plan-vs-actual refresh in minutes (tunable); capture never blocks
Scheduler scale	committed sequence for ≥10k open operations/tenant in interactive time
Offline capture	confirmations + handover queue locally, sync on reconnect (like M1)
Reconciliation	M3 throughput ties to M4 ProductionCount to the unit
Availability	99.5% serving (inherited)

Observability

OpenTelemetry traces on MRP run,
schedule run, confirm, handover sign-off

4. Acceptance criteria (module-level, gates v1 done)

- **MUST** honour posture end-to-end (A ingest-execute / B own-schedule / C system-of-record) with **no schema change** between them.
- **MUST** enforce finite capacity in committed schedules (no overload).
- **MUST** guarantee no orphaned order at handover; block sign-off on unacknowledged mandatory carryover.
- **MUST** book one ProductionCount per produced qty; reconcile with M4.
- **MUST** price lateness/idle/changeover via effective-dated cost rate (D7).
- **MUST** keep canon independent (no FK canon→ops; soft order_ref).
- **MUST** run hybrid/edge + offline-tolerant + tenant-isolated + audited.
- **SHOULD** reproduce the Hero Steels cold-rolling campaign sequence on the seed template.
- **SHOULD** bootstrap the full lifecycle from the M1 SAP-PP CSV + prod_* actuals with zero data loss.

5. Test plan (summary)

Unit: state-machine transitions, MRP gross-to-net arithmetic, setup-matrix costing, capacity math. Integration: M1 CSV→order→schedule→confirm→ProductionCount→KPI; SAP PP connector map. Cross-module: M3↔M4 count reconciliation; M2 PM window as capacity block. Acceptance: posture A/B/C walkthroughs; steel campaign; handover no-orphan + sign-off gating; offline capture sync. Schema: pglast.parse_sql in CI.

M3 • 10 — Glossary (Planning & Control)

APS (Advanced Planning & Scheduling) — finite-capacity planning/scheduling that considers all constraints (machine, labour, tooling, material) simultaneously to produce an executable schedule; contrast MRP (infinite capacity).

Attainment (schedule / quantity) — degree to which production matched the plan: schedule attainment = completed-on-schedule ÷ scheduled; quantity attainment = actual good ÷ planned.

BOM (Bill of Materials) — component structure of a material; exploded by MRP to derive component requirements.

Campaign (rolling) — a medium/long sequence of coils run together to minimise setups; steel campaigns sequence **wide**–**narrow** by width (and smoothly by gauge/hardness) to protect work rolls.

Capacity (RCCP / CRP) — Rough-Cut Capacity Planning validates the MPS against key resources; Capacity Requirements Planning checks detailed load at each work centre.

CRP / RCCP — see Capacity.

Confirmation — the execution actual booked against an order/operation (good/scrap/setup/run); books one canonical ProductionCount.

Dispatching (ISA-95) — releasing & assigning scheduled work to the floor (the dispatch list).

Finite vs infinite capacity — finite scheduling respects real resource limits; MRP assumes infinite capacity and emits planned orders, not executable schedules.

Handover (shift) — structured transfer of state between outgoing and incoming shifts; here, the no-orphan carryover + plan-vs-actual feedback that keeps the plan running 24×7.

Heijunka — Lean production levelling; smoothing volume/mix to stabilise flow (a sequencing input).

ISA-95 / IEC 62264 Part 3 — standard for Level-3 Manufacturing Operations Management activities: detailed scheduling, dispatching, execution, data collection, tracking, performance analysis (+ resource & definition management).

MPS (Master Production Schedule) — quantities & due dates for independent-demand end items; the source of demand for MRP.

MRP / MRP II — Material Requirements Planning explodes the BOM, nets against stock, offsets by lead time → planned orders; MRP II adds capacity, finance, etc.

OTIF (On-Time-In-Full) — orders delivered both on time and complete ÷ total orders.

Posture (planning) — Zedral's per-client planning depth: **A** adherence (ingest finished schedule, own execution), **B** own-the-schedule (ingest demand/MPS/MRP, own detailed scheduling), **C** system of record (own the whole hierarchy). Same model, different author.

Planned order — MRP's make/buy recommendation before release; firms then releases to a production order.

Production order — an authorised, released, costed job to make a quantity of a material by a date; the lifecycle spine. Its order_id is carried by canon.event.order_ref.

ProductionCount — canonical Event-Spine fact (good/scrap/rework qty); recorded once; M3 contextualises (attainment), M4 aggregates (OEE).

Routing — ordered operations (per work centre) to make a material; instantiated per order as production operations.

S&OP — Sales & Operations Planning; aggregate family-level plan above MPS (usually ERP/Level-4; out of M3 v1).

Sequence-dependent setup — changeover time/cost that depends on the order of jobs (captured in the setup matrix); the thing the optimiser minimises.

Setup matrix — table of transition costs from_state → to_state per work centre; encodes campaign rules (steel width bands, roll changes).

Throughput / WIP / cycle time — flow metrics: output per time; work in progress; release→completion elapsed.

What-if — candidate schedule under a scenario_tag, compared on KPIs before commit.

Appendix A • M3_schema.sql (runnable PostgreSQL 15+)

```
--
=====
-- Zedral · M3 Shopfloor Planning & Management — Reference Schema
(PostgreSQL 15+)
-- Status: v1 for build (plan -> schedule -> execute -> feed back).
-- Posture-configurable (ADHERENCE | SCHEDULING |
SYSTEM_OF_RECORD).
-- Optimisation/AI hooks present, build deferred (see deep-plan §8).
-- Conventions: follows canonical DDL spec (02_CanonicalDataModel_Spec §5):
-- lower_snake_case · surrogate BIGINT ids · UUID tenant_id · TIMESTAMPTZ
(UTC)
-- JSONB `ext` extension namespace · FK to canon.* · units embedded in column
name.
-- Assumes the canonical core already exists:
-- canon.equipment_node(node_id) -- ISA-95 hierarchy incl. WORK_CENTER &
WORK_UNIT
-- canon.event(event_id) -- Event Spine; production_count is a specialization
-- canon.shift(shift_id) -- shift/calendar (defines Planned Production Time)
-- canon.personnel(person_id) -- crew / supervisors
-- canon.cost_rate(rate_id ...) -- effective-dated rates (D7)
-- SOFT-REFERENCE RULE: canon.event.order_ref carries
ops.production_order.order_id
-- as a plain value (NO cross-schema hard FK) so canon stays independent of M3
--
-- the same independence rule M2 uses for downtime classification.
--
=====

CREATE SCHEMA IF NOT EXISTS ops;
SET search_path = ops, canon, public;

-----
-- 0. Enumerated types
-----
CREATE TYPE ops.planning_posture AS ENUM
('ADHERENCE','SCHEDULING','SYSTEM_OF_RECORD');
CREATE TYPE ops.material_type AS ENUM
('RAW','WIP','FINISHED','CONSUMABLE');
CREATE TYPE ops.demand_type AS ENUM
('FORECAST','CUSTOMER_ORDER','PIR','SAFETY_STOCK');
CREATE TYPE ops.order_source AS ENUM
('ERP_IMPORT','MRP_GENERATED','MANUAL','M1_PLAN_CSV');
CREATE TYPE ops.order_status AS ENUM
('PLANNED','FIRMED','RELEASED','SCHEDULED','DISPATCHED','IN_PROGRESS'
,'COMPLETED','CLOSED','CANCELLED');
CREATE TYPE ops.operation_status AS ENUM
```

```

('PENDING','DISPATCHED','IN_PROGRESS','CONFIRMED','SKIPPED');
CREATE TYPE ops.schedule_status AS ENUM
('PROPOSED','COMMITTED','SUPERSEDED');
CREATE TYPE ops.confirmation_type AS ENUM
('SETUP','PARTIAL','FINAL','SCRAP','REWORK');
CREATE TYPE ops.capacity_basis AS ENUM
('MACHINE_TIME','LABOR_TIME','THROUGHPUT_QTY','FURNACE_CHARGE');
CREATE TYPE ops.handover_status AS ENUM
('ACCUMULATING','DRAFTED','REVIEWED','SIGNED_OFF');
CREATE TYPE ops.carryover_kind AS ENUM
('OPEN_ORDER','WIP','STOPPAGE','QUALITY_HOLD','SAFETY','INSTRUCTION');
CREATE TYPE ops.mrp_run_type AS ENUM
('REGENERATIVE','NET_CHANGE');

```

```

-----
-- 1. Per-tenant planning configuration (the posture switch, §5)
-----

```

```

CREATE TABLE ops.planning_config (
  tenant_id      UUID PRIMARY KEY,
  posture        ops.planning_posture NOT NULL DEFAULT 'SCHEDULING',
  schedule_grain TEXT NOT NULL DEFAULT 'WORK_CENTER_SHIFT', --
  WORK_CENTER_SHIFT | MACHINE_MINUTE
  mrp_enabled    BOOLEAN NOT NULL DEFAULT FALSE,           -- TRUE only
  for SYSTEM_OF_RECORD
  optimiser_enabled BOOLEAN NOT NULL DEFAULT FALSE,       -- future
  scope (§8); gated by readiness
  ext           JSONB NOT NULL DEFAULT '{}':jsonb
);
COMMENT ON TABLE ops.planning_config IS 'Per-tenant planning posture;
selects which engine stages Zedral generates vs ingests (deep-plan §5).';
COMMENT ON COLUMN ops.planning_config.posture IS 'ADHERENCE=ingest
finished schedule; SCHEDULING=own detailed scheduling;
SYSTEM_OF_RECORD=own MPS/MRP.';

```

```

-----
-- 2. Material & BOM (canonical masters M3 brings to life)
-----

```

```

CREATE TABLE ops.material (
  material_id    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  tenant_id      UUID NOT NULL,
  sku            TEXT NOT NULL,
  name           TEXT NOT NULL,
  material_type  ops.material_type NOT NULL,
  base_uom       TEXT NOT NULL,           -- canonical base unit (e.g. 'MT',
  'EA')
  grade_ref      TEXT,                   -- maps to M1 grade master where present
  native_code    TEXT,
  source_ref     TEXT,
  ext           JSONB NOT NULL DEFAULT '{}':jsonb,

```

```

    UNIQUE (tenant_id, sku)
);
COMMENT ON TABLE ops.material IS 'Material/Product master
(raw/WIP/finished). Canonical entity M3 populates; FK target for orders & BOM.';

CREATE TABLE ops.bom_line (
    bom_line_id    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    tenant_id      UUID NOT NULL,
    parent_material_ref BIGINT NOT NULL REFERENCES
ops.material(material_id),
    component_material_ref BIGINT NOT NULL REFERENCES
ops.material(material_id),
    qty_per        NUMERIC NOT NULL CHECK (qty_per > 0),
    component_uom   TEXT NOT NULL,
    scrap_pct      NUMERIC NOT NULL DEFAULT 0 CHECK (scrap_pct >= 0),
    ext            JSONB NOT NULL DEFAULT '{}':jsonb
);
CREATE INDEX ix_bom_parent ON ops.bom_line (tenant_id, parent_material_ref);

```

-- 3. Routing & operations (how a material is made; the path orders follow)

```

CREATE TABLE ops.routing (
    routing_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    tenant_id      UUID NOT NULL,
    material_ref    BIGINT NOT NULL REFERENCES ops.material(material_id),
    version        INT NOT NULL DEFAULT 1,
    is_active      BOOLEAN NOT NULL DEFAULT TRUE,
    source_ref     TEXT,
    ext            JSONB NOT NULL DEFAULT '{}':jsonb,
    UNIQUE (tenant_id, material_ref, version)
);

CREATE TABLE ops.routing_operation (
    routing_operation_id BIGINT GENERATED BY DEFAULT AS IDENTITY
PRIMARY KEY,
    tenant_id      UUID NOT NULL,
    routing_ref     BIGINT NOT NULL REFERENCES ops.routing(routing_id),
    seq_no         INT NOT NULL,                -- order of operations in the route
    work_center_ref BIGINT NOT NULL REFERENCES
canon.equipment_node(node_id),
    operation_name  TEXT NOT NULL,              -- e.g. 'PICKLING',
'COLD_ROLLING', 'ANNEALING'
    setup_minutes_std NUMERIC NOT NULL DEFAULT 0,
    run_minutes_per_unit NUMERIC NOT NULL DEFAULT 0, -- standard run time
per base uom
    ext            JSONB NOT NULL DEFAULT '{}':jsonb,

```

```

    UNIQUE (tenant_id, routing_ref, seq_no)
);
CREATE INDEX ix_routeop_wc ON ops.routing_operation (tenant_id,
work_center_ref);

```

-- 4. Work-centre capacity & sequence-dependent setup (finite-capacity model, §6)

```

CREATE TABLE ops.work_center_capacity (
    capacity_id    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    tenant_id      UUID NOT NULL,
    work_center_ref BIGINT NOT NULL REFERENCES
    canon.equipment_node(node_id),
    shift_ref      BIGINT REFERENCES canon.shift(shift_id), -- capacity per shift
bucket (NULL = generic)
    basis          ops.capacity_basis NOT NULL DEFAULT 'MACHINE_TIME',
    available_minutes NUMERIC, -- available time in the bucket
    available_qty   NUMERIC, -- throughput-based capacity (e.g.
furnace charges)
    efficiency_pct  NUMERIC NOT NULL DEFAULT 100 CHECK (efficiency_pct >
0),
    effective_from  DATE NOT NULL,
    effective_to    DATE,
    ext            JSONB NOT NULL DEFAULT '{}':jsonb
);
CREATE INDEX ix_cap_wc ON ops.work_center_capacity (tenant_id,
work_center_ref, effective_from);

```

-- sequence-dependent setup/changeover cost matrix (steel: width wide->narrow, roll change)

```

CREATE TABLE ops.setup_matrix (
    setup_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    tenant_id      UUID NOT NULL,
    work_center_ref BIGINT NOT NULL REFERENCES
    canon.equipment_node(node_id),
    from_state     TEXT NOT NULL, -- attribute of outgoing job (e.g.
width band 'W1200')
    to_state       TEXT NOT NULL, -- attribute of incoming job
    setup_minutes  NUMERIC NOT NULL DEFAULT 0,
    setup_cost     NUMERIC, -- optional explicit cost
    is_forbidden   BOOLEAN NOT NULL DEFAULT FALSE, -- e.g. narrow->wide
w/o roll change forbidden
    rule_note      TEXT, -- 'width increase forces roll change'
    ext           JSONB NOT NULL DEFAULT '{}':jsonb,
    UNIQUE (tenant_id, work_center_ref, from_state, to_state)
);
COMMENT ON TABLE ops.setup_matrix IS 'Sequence-dependent setup
transitions; encodes steel campaign rules (wide->narrow). Substrate for the future

```


optimiser (\$8).';

-- 5. Demand & MPS (Posture C generates; A/B ingest)

```
CREATE TABLE ops.demand_forecast (  
  demand_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY  
KEY,  
  tenant_id      UUID NOT NULL,  
  material_ref    BIGINT NOT NULL REFERENCES ops.material(material_id),  
  demand_type    ops.demand_type NOT NULL,  
  qty            NUMERIC NOT NULL,  
  uom            TEXT NOT NULL,  
  demand_date    DATE NOT NULL,  
  customer_ref    TEXT,                -- maps to M1/canon customer master  
where present  
  source          ops.order_source NOT NULL DEFAULT 'ERP_IMPORT',  
  source_ref      TEXT,  
  ext            JSONB NOT NULL DEFAULT '{}'::jsonb  
);  
CREATE INDEX ix_demand_mat ON ops.demand_forecast (tenant_id,  
material_ref, demand_date);
```

```
CREATE TABLE ops.mps_line (  
  mps_id         BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY  
KEY,  
  tenant_id      UUID NOT NULL,  
  material_ref    BIGINT NOT NULL REFERENCES ops.material(material_id),  
  period_start    DATE NOT NULL,  
  period_end      DATE NOT NULL,  
  planned_qty     NUMERIC NOT NULL,  
  uom            TEXT NOT NULL,  
  source          ops.order_source NOT NULL DEFAULT 'MRP_GENERATED',  
  source_ref      TEXT,  
  ext            JSONB NOT NULL DEFAULT '{}'::jsonb  
);  
CREATE INDEX ix_mps_mat ON ops.mps_line (tenant_id, material_ref,  
period_start);
```

-- 6. MRP run log + on-hand stock (Posture C netting)

```
CREATE TABLE ops.mrp_run (  
  mrp_run_id     BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY  
KEY,  
  tenant_id      UUID NOT NULL,  
  run_type        ops.mrp_run_type NOT NULL DEFAULT 'NET_CHANGE',  
  started_ts      TIMESTAMPTZ NOT NULL DEFAULT now(),  
  finished_ts     TIMESTAMPTZ,
```

```

    planned_order_count INT,
    triggered_by BIGINT REFERENCES canon.personnel(person_id),
    ext JSONB NOT NULL DEFAULT '{}':jsonb
);

```

```

CREATE TABLE ops.material_stock (
    stock_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tenant_id UUID NOT NULL,
    material_ref BIGINT NOT NULL REFERENCES ops.material(material_id),
    site_ref BIGINT REFERENCES canon.equipment_node(node_id),
    on_hand_qty NUMERIC NOT NULL DEFAULT 0,
    allocated_qty NUMERIC NOT NULL DEFAULT 0,
    reorder_point NUMERIC,
    lead_time_days INT,
    as_of_ts TIMESTAMPTZ NOT NULL DEFAULT now(), -- source-owned in A/B; M3-owned in C
    source_ref TEXT,
    ext JSONB NOT NULL DEFAULT '{}':jsonb,
    UNIQUE (tenant_id, material_ref, site_ref)
);
COMMENT ON TABLE ops.material_stock IS 'On-hand / allocated balances for MRP netting (Posture C). In A/B these are read from ERP/WMS, not authored here.';

```

-- 7. Planned order (MRP output) -> Production order (released)

```

CREATE TABLE ops.planned_order (
    planned_order_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tenant_id UUID NOT NULL,
    material_ref BIGINT NOT NULL REFERENCES ops.material(material_id),
    mps_ref BIGINT REFERENCES ops.mps_line(mps_id),
    mrp_run_ref BIGINT REFERENCES ops.mrp_run(mrp_run_id),
    planned_qty NUMERIC NOT NULL,
    uom TEXT NOT NULL,
    need_date DATE NOT NULL,
    is_firmed BOOLEAN NOT NULL DEFAULT FALSE,
    source ops.order_source NOT NULL DEFAULT 'MRP_GENERATED',
    source_ref TEXT,
    ext JSONB NOT NULL DEFAULT '{}':jsonb
);
CREATE INDEX ix_plord_mat ON ops.planned_order (tenant_id, material_ref, need_date);

```

```

CREATE TABLE ops.production_order (
    order_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

```

```

tenant_id      UUID NOT NULL,
order_no       TEXT NOT NULL,           -- human/ERP order number
material_ref   BIGINT NOT NULL REFERENCES ops.material(material_id),
routing_ref    BIGINT REFERENCES ops.routing(routing_id),
planned_order_ref BIGINT REFERENCES ops.planned_order(planned_order_id),
order_status   ops.order_status NOT NULL DEFAULT 'PLANNED',
planned_qty    NUMERIC NOT NULL,
good_qty       NUMERIC NOT NULL DEFAULT 0,
scrap_qty      NUMERIC NOT NULL DEFAULT 0,
uom            TEXT NOT NULL,
priority       SMALLINT NOT NULL DEFAULT 5, -- 1=highest
due_ts        TIMESTAMPTZ,
release_ts     TIMESTAMPTZ,
completed_ts   TIMESTAMPTZ,
m1_coil_ref    TEXT,                   -- soft link to M1 COIL_NO spine where
applicable
source         ops.order_source NOT NULL DEFAULT 'ERP_IMPORT',
source_ref     TEXT,
ext            JSONB NOT NULL DEFAULT '{}':jsonb,
UNIQUE (tenant_id, order_no)
);
CREATE INDEX ix_po_status ON ops.production_order (tenant_id, order_status,
due_ts);
COMMENT ON COLUMN ops.production_order.order_id IS 'Value carried by
canon.event.order_ref (soft reference; no cross-schema hard FK).';

CREATE TABLE ops.production_operation (
operation_id   BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
tenant_id     UUID NOT NULL,
order_ref     BIGINT NOT NULL REFERENCES
ops.production_order(order_id),
routing_operation_ref BIGINT REFERENCES
ops.routing_operation(routing_operation_id),
seq_no        INT NOT NULL,
work_center_ref BIGINT NOT NULL REFERENCES
canon.equipment_node(node_id),
operation_status ops.operation_status NOT NULL DEFAULT 'PENDING',
planned_qty    NUMERIC,
good_qty       NUMERIC NOT NULL DEFAULT 0,
ext            JSONB NOT NULL DEFAULT '{}':jsonb,
UNIQUE (tenant_id, order_ref, seq_no)
);
CREATE INDEX ix_prodop_wc ON ops.production_operation (tenant_id,
work_center_ref, operation_status);

```

-- 8. Detailed schedule (finite-capacity sequence) + dispatch (M3b)

```

CREATE TABLE ops.schedule_entry (

```

```

    sched_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    tenant_id     UUID NOT NULL,
    order_ref     BIGINT NOT NULL REFERENCES
ops.production_order(order_id),
    operation_ref BIGINT REFERENCES ops.production_operation(operation_id),
    work_center_ref BIGINT NOT NULL REFERENCES
canon.equipment_node(node_id),
    shift_ref     BIGINT REFERENCES canon.shift(shift_id),
    sequence_no   INT NOT NULL,           -- position in the work-centre
sequence
    planned_start_ts TIMESTAMPTZ,
    planned_end_ts   TIMESTAMPTZ,
    setup_minutes    NUMERIC NOT NULL DEFAULT 0,   -- from setup_matrix
given the predecessor
    status           ops.schedule_status NOT NULL DEFAULT 'PROPOSED',
    scenario_tag     TEXT,               -- what-if scenario name; NULL =
committed plan
    ext              JSONB NOT NULL DEFAULT '{}':jsonb
);
CREATE INDEX ix_sched_wc_seq ON ops.schedule_entry (tenant_id,
work_center_ref, sequence_no);
CREATE INDEX ix_sched_order ON ops.schedule_entry (tenant_id, order_ref);

-----
-- 9. Production confirmation (actuals -> books a canon ProductionCount, §12)
-----
CREATE TABLE ops.production_confirmation (
    conf_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    tenant_id     UUID NOT NULL,
    order_ref     BIGINT NOT NULL REFERENCES
ops.production_order(order_id),
    operation_ref BIGINT REFERENCES ops.production_operation(operation_id),
    conf_type     ops.confirmation_type NOT NULL DEFAULT 'PARTIAL',
    work_center_ref BIGINT REFERENCES canon.equipment_node(node_id),
    shift_ref     BIGINT REFERENCES canon.shift(shift_id),
    operator_ref  BIGINT REFERENCES canon.personnel(person_id),
    good_qty      NUMERIC NOT NULL DEFAULT 0,
    scrap_qty     NUMERIC NOT NULL DEFAULT 0,
    rework_qty    NUMERIC NOT NULL DEFAULT 0,
    setup_minutes NUMERIC NOT NULL DEFAULT 0,   -- actual setup (calibrates
setup_matrix / optimiser)
    run_minutes   NUMERIC NOT NULL DEFAULT 0,
    confirmed_ts  TIMESTAMPTZ NOT NULL DEFAULT now(),
    canon_event_ref BIGINT,           -- soft link to canon.event (the
ProductionCount row)
    m1_source_ref TEXT,               -- provenance to M1 prod_* capture
    source_ref    TEXT,
    ext           JSONB NOT NULL DEFAULT '{}':jsonb

```

```

);
CREATE INDEX ix_conf_order ON ops.production_confirmation (tenant_id,
order_ref, confirmed_ts);
COMMENT ON TABLE ops.production_confirmation IS 'Execution actuals. The
good/scrap qty is recorded once as a canon ProductionCount; M3 contextualises
(attainment), M4 aggregates (OEE). No double count (§12).';

-----
-- 10. Shift handover (M3a) — header, carryover lines, notes
-----

CREATE TABLE ops.shift_handover (
  handover_id    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
  tenant_id      UUID NOT NULL,
  site_ref       BIGINT REFERENCES canon.equipment_node(node_id),
  area_ref       BIGINT REFERENCES canon.equipment_node(node_id),
  shift_out_ref   BIGINT REFERENCES canon.shift(shift_id),
  shift_in_ref    BIGINT REFERENCES canon.shift(shift_id),
  outgoing_supervisor_ref BIGINT REFERENCES canon.personnel(person_id),
  incoming_supervisor_ref BIGINT REFERENCES canon.personnel(person_id),
  status         ops.handover_status NOT NULL DEFAULT 'ACCUMULATING',
  planned_qty     NUMERIC,                -- shift plan vs ...
  produced_qty    NUMERIC,                -- ... shift actual (plan-vs-actual
feedback)
  oee_pct         NUMERIC,                -- shift OEE snapshot (from M4)
  variance_note   TEXT,
  drafted_ts      TIMESTAMPTZ,
  signed_off_ts   TIMESTAMPTZ,
  ext            JSONB NOT NULL DEFAULT '{}':jsonb
);
CREATE INDEX ix_handover_shift ON ops.shift_handover (tenant_id,
shift_out_ref);
COMMENT ON TABLE ops.shift_handover IS 'Graduates M1 first-class handover
to the platform; the no-orphan guarantee + plan-vs-actual feedback into planning
(deep-plan §7).';

CREATE TABLE ops.handover_carryover (
  carryover_id    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
  tenant_id      UUID NOT NULL,
  handover_ref    BIGINT NOT NULL REFERENCES
ops.shift_handover(handover_id),
  kind           ops.carryover_kind NOT NULL,
  order_ref       BIGINT REFERENCES ops.production_order(order_id),  -- open
order / WIP
  asset_ref       BIGINT REFERENCES canon.equipment_node(node_id),  --
degraded asset / stoppage
  description     TEXT NOT NULL,
  is_mandatory    BOOLEAN NOT NULL DEFAULT FALSE, -- safety/exception
items that cannot be skipped

```

```
    acknowledged    BOOLEAN NOT NULL DEFAULT FALSE, -- incoming shift  
acknowledged
```

```
    ext              JSONB NOT NULL DEFAULT '{}':jsonb  
);
```

```
CREATE INDEX ix_carryover_handover ON ops.handover_carryover (tenant_id,  
handover_ref);
```

```
-- -----  
-- 11. KPI rollout mart (served to UIL / Reporting; reconciles with M4)  
-----
```

```
CREATE TABLE ops.kpi_plan_daily (  
    kpi_id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    tenant_id       UUID NOT NULL,  
    work_center_ref BIGINT REFERENCES canon.equipment_node(node_id),  
    kpi_date        DATE NOT NULL,  
    schedule_attainment_pct NUMERIC,  
    qty_attainment_pct    NUMERIC,  
    otif_pct            NUMERIC,  
    throughput_qty      NUMERIC,  
    capacity_util_pct   NUMERIC,  
    setup_ratio_pct     NUMERIC,  
    changeover_count    INT,  
    wip_avg_qty         NUMERIC,  
    late_order_cost     NUMERIC, -- priced via canon.cost_rate effective  
at date (D7)  
    idle_capacity_cost   NUMERIC,  
    ext                 JSONB NOT NULL DEFAULT '{}':jsonb,  
    UNIQUE (tenant_id, work_center_ref, kpi_date)  
);
```

```
COMMENT ON TABLE ops.kpi_plan_daily IS 'Daily planning-KPI mart. OEE-  
adjacent figures reconcile to M4 (same ProductionCount truth).';
```

```
-- -----  
-- 12. Convenience view: committed dispatch list per work centre / shift  
-----
```

```
CREATE VIEW ops.v_dispatch_list AS  
SELECT s.tenant_id,  
    s.work_center_ref,  
    s.shift_ref,  
    s.sequence_no,  
    s.order_ref,  
    po.order_no,  
    po.material_ref,  
    s.planned_start_ts,  
    s.planned_end_ts,  
    s.setup_minutes,  
    po.priority  
FROM ops.schedule_entry s  
JOIN ops.production_order po ON po.order_id = s.order_ref  
WHERE s.status = 'COMMITTED' AND s.scenario_tag IS NULL
```

ORDER BY s.tenant_id, s.work_center_ref, s.shift_ref, s.sequence_no;

```
--
=====
=====
-- End M3 reference schema. ~20 tables + 1 view, ops schema, FK to canon.*.
-- Optimisation/AI (deep-plan §8) reuses setup_matrix + work_center_capacity
-- + production_confirmation actuals — no re-model required to switch it on.
--
=====
=====
```